MoSAIC: Modular Self-Attentive Interacting Columns for Recurrent Memory

Anonymous submission

Abstract

Recurrent models preserve a fixed-size state, but conventionally organize that state as one monolithic stream per layer. We introduce MoSAIC (Modular Self-Attentive Interacting Columns), which factorizes recurrent state into persistent GRU columns and uses attention-mediated routing to determine the input received by each column. Under approximately parameter-matched, one-billion-token training, this structural bias consistently improves both associative memory and character-level prediction. On Store–Distract–Query, MoSAIC-L2C4 obtains 0.843 ± 0.006 final-window accuracy on long-gap queries versus 0.532 ± 0.103 for a two-layer GRU; the best observed configuration, L3C4, obtains 0.920 ± 0.013. On text8, L2C4 obtains 1.437 ± 0.003 validation bits per character (BPC) and L3C4 obtains 1.435 ± 0.002, compared with 1.483 ± 0.006 for the strongest GRU and 1.449 ± 0.012 for a similarly sized finite-context Transformer. These results support modular state organization as a useful inductive bias for recurrent computation at this scale.

Introduction

Recurrent neural networks compress a sequence history into a fixed-size state and update it incrementally. This property makes them natural models for streaming prediction and memory tasks. A conventional GRU (Cho et al. 2014) or LSTM (Hochreiter and Schmidhuber 1997), however, represents each layer as a single hidden-state stream. Increasing its width or depth adds capacity, but leaves the internal organization of state and the exchange of information within that state implicit.

We ask whether a fixed recurrent parameter budget can be organized into persistent modules whose communication is learned. Rather than extending nominal context length, the goal is an explicit recurrent topology: separate state-bearing components maintain local dynamics, while content-dependent routing moves information among them.

We introduce *MoSAIC (Modular Self-Attentive Interacting Columns)*. MoSAIC distributes each recurrent layer across independently parameterized GRU columns. Before every recurrent update, the previous state of each column queries a small message bank through multi-head attention. The bottom layer reads the current input together with the delayed top-layer states from the preceding timestep; higher layers read the newly updated states below them. All columns update at every timestep, and the complete top layer persists to the next timestep.

This design uses modular state organization as an inductive bias rather than sparse expert selection. The number of columns and layers provides explicit axes along which recurrent state and computation can be organized. Attention is applied only over a fixed collection of columns and current inputs, so the recurrent state remains fixed in size as the processed sequence grows.

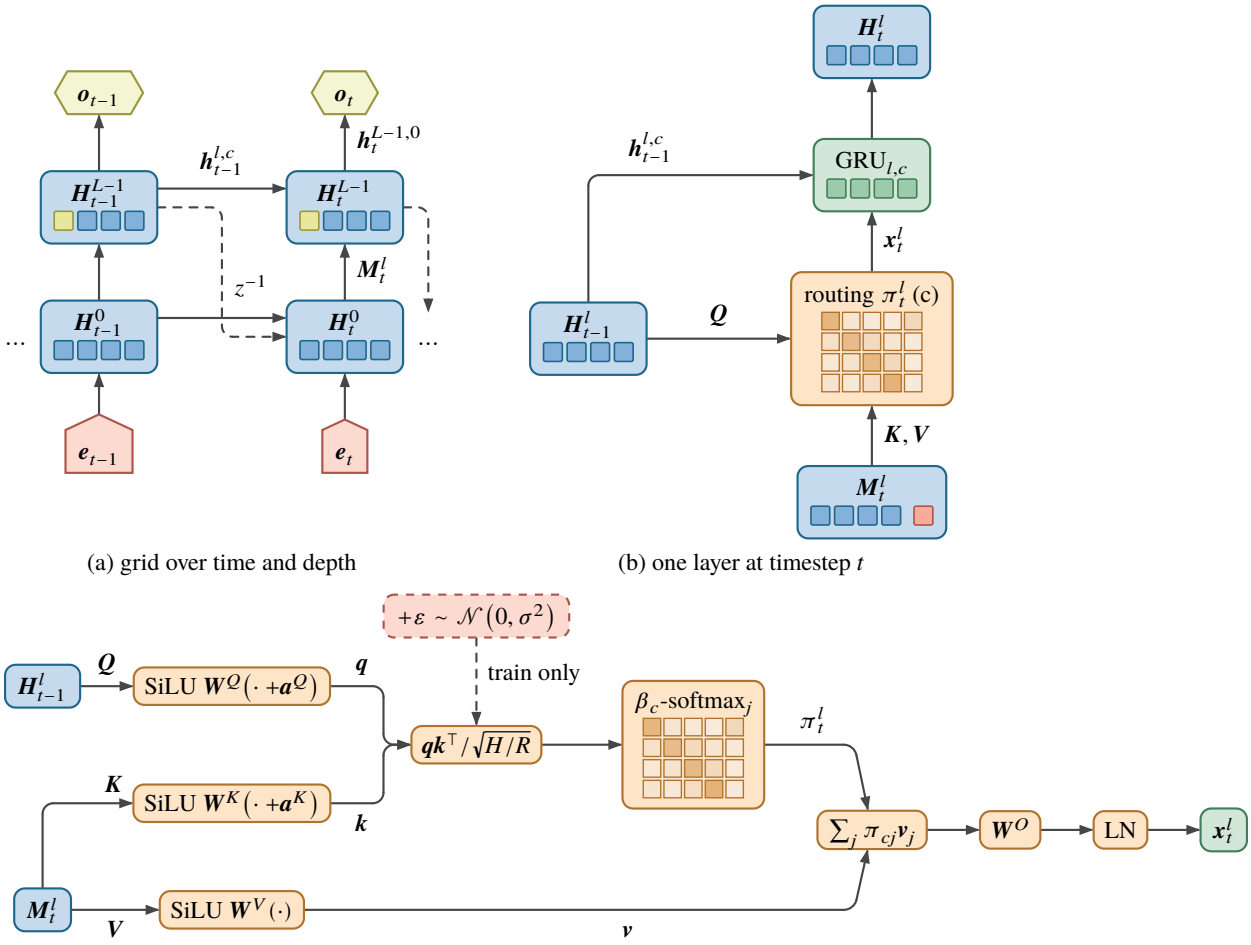
We evaluate at approximately ten million parameters on Store–Distract–Query (SDQ) and text8. Principal comparisons use a common one-billion-token horizon and multiple completed runs; text8 also includes similarly sized finite-context Transformers. L2C4 is a compact reference shape and L3C4 the best observed configuration, without designating either as a universal primary model.

Our contributions are:

- **Architecture:** a layered grid of persistent recurrent columns with attention-mediated inter-column communication.
- **Controlled topology:** column count and depth provide explicit axes for organizing recurrent state and computation at roughly fixed parameter scale.
- **Empirical evidence:** matched GRU/MoSAIC comparisons show consistent improvements on SDQ and text8, and the text8 comparison includes a similarly sized finite-context Transformer.

Related Work

Modular recurrence and communication. Recurrent Independent Mechanisms (RIMs) (Goyal et al. 2021) maintain separate recurrent modules, sparsely activate a subset for each input, and allow active modules to communicate through attention. BRIMs (Mittal et al. 2020) add hierarchical bottom-up and top-down communication, while shared-workspace models (Goyal et al. 2022) impose a communication bottleneck. Relational Memory Core (Santoro et al. 2018) instead updates interacting memory slots through attention. These works establish modular recurrence and learned communication as important design principles. MoSAIC differs in using a regular layered topology with dense synchronous updates: every persistent column updates at every timestep after reading from a layer-specific message bank.



(a) grid over time and depth

(b) one layer at timestep t

(c) inside the router of panel (b)

Figure 1: MoSAIC mechanism. At timestep t, each persistent column state queries a layer-specific message bank. The first layer reads the current input together with the previous top-layer column states; later layers read the newly updated states below them. The routed message is the input to an independent recurrent update. All top-layer column states are carried to timestep $t + 1$.

We therefore describe the mechanism as learned routing, not sparse or conditional computation.

Grid and multidimensional recurrence. Multi-Dimensional RNNs (Graves, Fernandez, and Schmidhuber 2007) propagate information recurrently along multiple axes of structured data. Grid LSTM (Kalchbrenner, Danihelka, and Graves 2016) arranges LSTM blocks in a multidimensional grid whose dimensions exchange hidden and memory vectors through fixed connections. MoSAIC uses a grid as an organization of modules rather than as an additional axis of the input. All columns share one temporal axis, retain separate recurrent states, and communicate through content-dependent routing rather than edges fixed by input geometry.

Modern recurrent and attention models. Recent architectures strengthen recurrent alternatives to full self-attention by redesigning the recurrent update or its memory representation. LRUs (Orvieto et al. 2023) stabilize long linear recurrences; HGRN2 (Qin et al. 2024) expands the state of a gated linear RNN; Mamba-2 (Dao and Gu 2024) connects selective state-space models with structured attention; DeltaNet (Yang et al. 2024) uses delta-rule matrix state; and xLSTM (Beck et al. 2024) includes scalar- and matrix-memory variants. Transformers (Vaswani et al. 2017) and segment-recurrent variants such as Transformer-XL (Dai et al. 2019) instead retain a finite or growing bank of token representations for attention. MoSAIC keeps a conventional nonlinear GRU update and changes how recurrent state is divided and routed. Our experiments compare against GRUs and finite-context Transformers under the same token budget; memory-limited implementations of HGRN2, DeltaNet, and mLSTM are reported only under their separate reduced-token protocols.

Method

Persistent Columns and Layered Topology

Let $\mathbf{e}_t \in \mathbb{R}^{B\times H}$ be an embedded input for a batch of size B. A conventional L-layer GRU maintains $\mathbf{S}_t \in \mathbb{R}^{L\times B\times H}$. MoSAIC instead maintains

$$\mathbf{H}_t \in \mathbb{R}^{L\times C\times B\times H}, \tag{1}$$

where $\mathbf{h}_t^{l,c}$ denotes column c at layer l. Each layer–column pair has independent GRU parameters. Columns in a layer are evaluated together for efficiency, but do not share recurrent weights.

The message bank for the first layer contains the delayed top layer and the current external inputs:

$$\mathbf{M}_t^0 = [\mathbf{H}_{t-1}^{L-1}; \mathbf{e}_t^1; \ldots; \mathbf{e}_t^I] \in \mathbb{R}^{(C+I)\times B\times H}. \tag{2}$$

Here I is the number of input streams and $I = 1$ in our experiments. At subsequent layers,

$$\mathbf{M}_t^l = \mathbf{H}_t^{l-1}, \qquad l > 0. \tag{3}$$

Information therefore moves bottom-up within a timestep, while the complete top layer provides a delayed feedback path across timesteps. Figure 1 summarizes this computation.

Attention-Mediated Routing

At layer l, the previous states of its C columns form the queries. Learnable identities distinguish query columns and message sources. For head r,

$$\mathbf{q}_{t,c,r}^l = \mathrm{SiLU}\Big(\mathbf{W}_r^Q(\mathbf{h}_{t-1}^{l,c} + \mathbf{a}_c^Q) + \mathbf{b}_r^Q\Big), \tag{4}$$

$$\mathbf{k}_{t,j,r}^l = \mathrm{SiLU}\big(\mathbf{W}_r^K(\mathbf{m}_{t,j}^l + \mathbf{a}_j^K) + \mathbf{b}_r^K\big), \tag{5}$$

$$\mathbf{v}_{t,j,r}^l = \mathrm{SiLU}\big(\mathbf{W}_r^V\,\mathbf{m}_{t,j}^l + \mathbf{b}_r^V\big). \tag{6}$$

Each query column has a learned positive inverse temperature $\beta_c = \mathrm{softplus}(\rho_c)$. The routing probabilities are

$$\pi_{t,c,j,r}^l = \mathrm{softmax}_j\left[\beta_c\left(\frac{\mathbf{q}_{t,c,r}^l \cdot \mathbf{k}_{t,j,r}^l}{\sqrt{H/R}} + \epsilon_{t,c,j,r}^l\right)\right], \tag{7}$$

where R is the number of heads. The perturbation ϵ is independent zero-mean Gaussian noise during training and zero at evaluation. Concatenated head outputs are linearly projected and layer-normalized:

$$\begin{aligned}\mathbf{z}_{t,c,r}^l &= \sum_j \pi_{t,c,j,r}^l \mathbf{v}_{t,j,r}^l, && r = 1,\ldots,R,\\ \mathbf{x}_{t,c}^l &= \mathrm{LN}\big(\mathbf{W}^O[\mathbf{z}_{t,c,1}^l; \ldots; \mathbf{z}_{t,c,R}^l]\big).\end{aligned} \tag{8}$$

Routing determines the input to each recurrent cell; it does not replace or post-process its persistent state.

Recurrent Update and Readout

Every column performs an ordinary independent GRU update,

$$\mathbf{h}_t^{l,c} = \mathrm{GRU}_{l,c}(\mathbf{x}_{t,c}^l, \mathbf{h}_{t-1}^{l,c}). \tag{9}$$

The prediction head reads column 0 of the top layer. All top-layer columns remain in the recurrent state and form part of the next bottom-layer message bank. This makes routing necessary for information stored outside the readout column to affect predictions.

The main configurations additionally use three training-only routing terms: Gaussian logit noise, a communication cost favoring diagonal routes, and an entropy bonus that discourages early route collapse. Let S_l be the number of messages available at layer l. With zero-based column index c, the route cost $d_{c,j}^l$ is zero for the same-column recurrent message, one for another recurrent message, and $2c$ for the external input in the bottom layer. The communication loss is the mean expected route cost,

$$\mathcal{L}_{\mathrm{comm}} = \mathbb{E}_{t,l,c,r}\left[\sum_{j=1}^{S_l} \pi_{t,c,j,r}^l d_{c,j}^l\right]. \tag{10}$$

Routing entropy is normalized by the log of the number of available messages,

$$\overline{\mathcal{H}}(\pi) = \mathbb{E}_{t,l,c,r}\left[\frac{-\sum_{j=1}^{S_l} \pi_{t,c,j,r}^l \log \pi_{t,c,j,r}^l}{\log S_l}\right]. \tag{11}$$

For task loss $\mathcal{L}_{\mathrm{task}}$,

$$\mathcal{L} = \mathcal{L}_{\mathrm{task}} + \lambda_{\mathrm{comm}}\mathcal{L}_{\mathrm{comm}} - \lambda_{\mathrm{ent}}\overline{\mathcal{H}}(\pi). \tag{12}$$

All MoSAIC configurations use four routing heads. For text8, $(\lambda_{\mathrm{comm}}, \lambda_{\mathrm{ent}}, \sigma_\epsilon) = (0.05, 0.005, 0.05)$; for SDQ, the settings are $(0.02, 0.002, 0.02)$. These terms add no inference-time state and are not treated as separate empirical contributions in the absence of completed component ablations.

Fixed-State and Resource Accounting

Attention is applied over $C + I$ messages in the bottom layer and C messages above it, rather than over token history. For fixed L, C, and H, MoSAIC therefore carries a fixed-size recurrent state and supports incremental processing as sequence length grows. This is an architectural property, not an empirical claim of superior long-context scaling.

Ignoring embeddings and biases, the independent GRUs contribute approximately $6LCH^2$ parameters and routing projections approximately $4LH^2$. Persistent state contains LCH scalars per example, compared with LH for a stacked GRU. Parameter matching consequently does not match activation state or computation. We report recurrent-state size to make this distinction explicit, but do not claim resource efficiency.

Experiments

Shared Protocol and Reporting

All principal GRU and MoSAIC configurations are approximately parameter matched at ten million trainable parameters. The principal horizon is one billion processed tokens. Runs with a final logged evaluation point slightly below one billion are retained only under the documented logging-loss convention; longer runs are truncated at the one-billion-token horizon. Results report the mean and standard deviation across completed replicates. We make no statistical-significance claim.

Family	Shape	Params	n	Endpoint	Final-five Acc++ ↑
GRU	L1	10.18M	3	970M	0.6177 ± 0.0052
GRU	L2	10.06M	2	980M	0.5322 ± 0.1027
GRU	L3	10.25M	2	990M	0.2865 ± 0.0798
MoSAIC	L1C8	10.12M	2	1000M	0.7116 ± 0.0167
MoSAIC	L2C4	10.12M	3	985M	0.8433 ± 0.0055
MoSAIC	L2C8	10.18M	1	1000M	0.8678
MoSAIC	L2C16	10.09M	1	1000M	0.8901
MoSAIC	L3C4	9.99M	3	1000M	$\mathbf{0.9204 \pm 0.0128}$

Table 1: Matched SDQ results under the one-billion-token target. "Endpoint" is the final logged point used for each configuration under the logging-loss convention. Values are mean ± standard deviation of replicate-level final-five Acc++ averages.

L2C4 is the compact reference configuration used for direct two-layer comparisons. L3C4 is the best observed configuration in both tasks. The shape sweep is reported in full rather than using rhetoric to designate either as a universal primary model.

Store–Distract–Query

Task. SDQ is an online synthetic task designed to isolate associative storage and retrieval under interference. With five keys and ten values, the vocabulary comprises 50 key–value store tokens, ten distractor tokens, and five query tokens. A store overwrites the value associated with its key. Distractor values contribute to a running sum modulo ten. At a query, the target combines the current stored value, the distractor sum, and the configured per-key store and query counts. Cross-entropy is applied only at query events.

Episode lengths are geometrically distributed. During training, their mean increases from 10 toward 500, while store and query probabilities decrease from 0.35 toward 0.10 and 0.25. Models use 512 parallel streams, a truncation length of 32, RMSprop, gradient clipping at norm 1, and the same learning-rate schedule and online generator.

Metric. The code logs Acc++ as query accuracy on examples whose valid store–query gaps are above the current batch mean. This emphasizes the longer-gap half of non-missing queries. For each completed replicate, we average the final five logged Acc++ values through the one-billion-token horizon and then aggregate those replicate-level averages.

Results. Table 1 shows a consistent family-level advantage for MoSAIC: every evaluated shape has a higher central Acc++ value than every GRU depth. In the replicated two-layer comparison, L2C4 reaches 0.8433 ± 0.0055, while GRU-L2 reaches 0.5322 ± 0.1027. L3C4 is the best observed configuration at 0.9204 ± 0.0128. The single-replicate L2C8 and L2C16 entries describe observed runs but provide less evidence about run-to-run variability.

Character-Level Modeling

Data and training. Text8 (Mahoney 2011) contains 100 million normalized English characters with a 27-character alphabet. We use the first 90 million characters for training and the final 10 million as a contiguous validation split. The validation split was used for limited manual configuration tuning, so we call the metric validation BPC rather than an untouched test estimate.

GRU and MoSAIC models process 512 parallel contiguous streams with truncated backpropagation through time over 64 steps. Random state resets decay from probability $0.2/64$ toward $0.001/64$ per token, increasing the expected uninterrupted context from about 320 to 64,000 characters. Models use RMSprop, a learning rate warming to 5×10^{-4} and decaying toward 5×10^{-5}, and gradient clipping at norm 1. Validation runs disable random resets and evaluate contiguous streams.

The Transformer uses the same tokenization, data split, reset schedule, optimizer, token batch per update, and one-billion-token budget. It has four layers, hidden width 560, four attention heads, feed-forward width 1120, RMSNorm, and rotary position embeddings (RoPE). It processes each 64-token training segment in parallel and carries a rolling valid key/value cache of 256 or 64 tokens between segments; the two cache lengths are distinct configurations.

Results. As shown in Table 2, every evaluated MoSAIC shape has a lower central BPC than every matched GRU. The compact L2C4 reference reaches 1.4367 ± 0.0026, compared with 1.5004 ± 0.0119 for GRU-L2. L3C4 is the best observed shape at 1.4345 ± 0.0017; the strongest GRU, L3, reaches 1.4828 ± 0.0062. The cache-256 Transformer reaches 1.4492 ± 0.0120 under the same token budget. Thus the observed MoSAIC means are lower while retaining recurrent fixed-state operation; this comparison does not establish general Transformer superiority or long-context scaling.

Topology and State Allocation

Parameter matching changes hidden width and persistent-state size. Table 3 isolates those differences for the recurrent families. At two layers, increasing MoSAIC from four to eight and sixteen columns increases state size while worsening text8 BPC relative to L2C4. L2C16 stores more than twice as many recurrent scalars as L2C4 but obtains a higher BPC. On SDQ, the central values increase across these shapes, but L2C8 and L2C16 each have only one completed run. Depth also behaves differently across families: L3C4 improves over L2C4 on both tasks, whereas deeper GRUs improve on text8 but not SDQ. These observations show that state volume

Family	Shape / cache	Params	n	Endpoint	Validation BPC ↓
GRU	L1	10.16M	3	980M	1.5626 ± 0.0028
GRU	L2	10.04M	3	1000M	1.5004 ± 0.0119
GRU	L3	10.23M	3	980M	1.4828 ± 0.0062
MoSAIC	L1C8	10.11M	2	1000M	1.4709 ± 0.0033
MoSAIC	L2C4	10.11M	3	980M	1.4367 ± 0.0026
MoSAIC	L2C8	10.17M	2	1000M	1.4444 ± 0.0017
MoSAIC	L2C16	10.09M	2	1000M	1.4548 ± 0.0023
MoSAIC	L3C4	9.99M	3	1000M	$\mathbf{1.4345 \pm 0.0017}$
Transformer	cache 256	10.07M	3	1000M	1.4492 ± 0.0120
Transformer	cache 64	10.07M	3	962.6M	1.4826 ± 0.0057

Table 2: Fixed-horizon text8 results under the one-billion-token target. "Endpoint" is the final logged validation point used for each configuration. The Transformer cache variants are distinct configurations, not replicates.

Shape	H	State	Params
GRU-L1	1,296	1,296	10.16M
GRU-L2	912	1,824	10.04M
GRU-L3	752	2,256	10.23M
L1C8	440	3,520	10.11M
L2C4	424	3,392	10.11M
L2C8	312	4,992	10.17M
L2C16	224	7,168	10.09M
L3C4	344	4,128	9.99M

Table 3: Recurrent topology and persistent-state scalars per example for text8 configurations. SDQ uses the same hidden widths and shapes, with small parameter-count differences from its input and output vocabularies.

Model	Tokens	n	Updates	Final BPC ↓
HGRN2	100M	2	48.8k	1.6675 ± 0.0076
DeltaNet	200M	2	48.8k	1.8280 ± 0.0231
mLSTM	200M	1	48.8k	1.6856

Table 4: Reduced-token text8 baselines. HGRN2 uses 2,048 tokens/update; DeltaNet and mLSTM use 4,096. Standard GRU, MoSAIC, and Transformer runs use 32,768 tokens/update and approximately 30.5k updates.

alone does not explain the results and that the allocation across depth, column count, and within-column width matters.

Reduced-Token Baselines

HGRN2, DeltaNet, and mLSTM exceed the available accelerator memory at the standard 512-stream geometry. They were therefore trained with fewer streams, smaller token batches per update, reduced token budgets, and more optimizer updates than the one-billion-token runs. Table 4 reports these runs as a separate implementation-level reference. They are not equal-token comparisons and do not support claims about relative resource or optimization efficiency.

Discussion and Limitations

Across the controlled comparisons, MoSAIC's columnar topology is associated with better associative-memory and character-modeling results than monolithic recurrent baselines at approximately fixed parameter scale. The pattern is consistent across the evaluated shapes rather than depending only on the best run family member. The text8 result relative to the finite-context Transformer provides additional context: MoSAIC retains recurrent fixed-state operation and obtains a lower mean BPC under the present matched token budget.

These results support a narrow conclusion. They do not show that columns acquire identifiable functional roles, that dense routing behaves as sparse expert selection, or that MoSAIC replaces Transformers generally. The architecture permits incremental inference with state size independent of processed sequence length, but the experiments do not establish superior long-context scaling or long-horizon retention against all alternatives.

Parameter matching also leaves important resources unmatched. MoSAIC exposes more persistent state coordinates than the GRUs and performs routing computation at every layer and timestep. We have not completed controlled throughput, peak-memory, or inference-latency measurements, so the results should be interpreted as evidence about an architectural inductive bias rather than resource efficiency. Likewise, the reduced-token recurrent baselines use different token and update budgets and are not direct quality comparisons.

The empirical scope is limited. Text8 is a small character-level corpus, its validation split received limited manual tuning, and the study operates at roughly ten million parameters. SDQ is synthetic and uses a curriculum-controlled online generator. Some topology entries have only one or two completed runs, and we do not make statistical-significance claims. Training-only routing regularizers add hyperparameters whose individual effects have not been isolated. Finally, the present paper omits reinforcement-learning results because the task-matched evidence is incomplete and variable.

Conclusion

MoSAIC factorizes a monolithic recurrent state into persistent columns and learns how information is routed among them before independent recurrent updates. Under controlled, approximately parameter-matched one-billion-token training, this structural bias consistently improves associative memory and character-level modeling over monolithic GRUs. A similarly sized finite-context Transformer provides a matched text8 reference, while the separate reduced-token results are reported only as non-comparable implementation context. The evidence supports modular state organization as a useful inductive bias for recurrent computation at this scale; broader claims about specialization, long-context scaling, and resource efficiency remain open.

References

Beck, M.; Pöppel, K.; Spanring, M.; Auer, A.; Prudnikova, O.; Kopp, M.; Klambauer, G.; Brandstetter, J.; and Hochreiter, S. 2024. xLSTM: Extended Long Short-Term Memory. *arXiv preprint arXiv:2405.04517*.

Cho, K.; van Merrienboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; and Bengio, Y. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734. Association for Computational Linguistics.

Dai, Z.; Yang, Z.; Yang, Y.; Carbonell, J.; Le, Q. V.; and Salakhutdinov, R. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2978–2988.

Dao, T.; and Gu, A. 2024. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. *arXiv preprint arXiv:2405.21060*.

Goyal, A.; Didolkar, A.; Lamb, A.; Badola, K.; Ke, N. R.; Rahaman, N.; Binas, J.; Blundell, C.; Mozer, M.; and Bengio, Y. 2022. Coordination Among Neural Modules Through a Shared Global Workspace. In *10th International Conference on Learning Representations (ICLR)*.

Goyal, A.; Lamb, A.; Hoffmann, J.; Sodhani, S.; Levine, S.; Bengio, Y.; and Schölkopf, B. 2021. Recurrent Independent Mechanisms. In *9th International Conference on Learning Representations (ICLR)*.

Graves, A.; Fernandez, S.; and Schmidhuber, J. 2007. Multi-Dimensional Recurrent Neural Networks. In *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN)*, 549–558.

Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.

Kalchbrenner, N.; Danihelka, I.; and Graves, A. 2016. Grid Long Short-Term Memory. In *3rd International Conference on Learning Representations (ICLR)*.

Mahoney, M. 2011. About the Test Data. https://www.mattmahoney.net/dc/textdata.html. Text8 dataset description.

Mittal, S.; Lamb, A.; Goyal, A.; Voleti, V.; Shanahan, M.; Lajoie, G.; Mozer, M.; and Bengio, Y. 2020. Learning to Combine Top-Down and Bottom-Up Signals in Recurrent Neural Networks with Attention over Modules. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 6972–6986.

Orvieto, A.; Smith, S. L.; Gu, A.; Fernando, A.; Gulcehre, C.; Pascanu, R.; and De, S. 2023. Resurrecting Recurrent Neural Networks for Long Sequences. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 26670–26698.

Qin, Z.; Yang, S.; Sun, W.; Shen, X.; Li, D.; Li, W.; and Zheng, Y. 2024. HGRN2: Gated Linear RNNs with State Expansion. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Santoro, A.; Faulkner, R.; Raposo, D.; Rae, J.; Chrzanowski, M.; Weber, T.; Wierstra, D.; Vinyals, O.; Pascanu, R.; and Lillicrap, T. 2018. Relational Recurrent Neural Networks. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 7310–7321.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Yang, S.; Wang, B.; Zhang, Y.; Shen, Y.; and Kim, Y. 2024. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. *arXiv preprint arXiv:2406.06484*.